VisionSecure: A Real-Time Screen Privacy Protection System

Using Digital Image Processing Techniques for Live PII Redaction

Sabhya Gupta and Prakshal Choudhary
23BCE2362 and 23BCE2268
Digital Image Processing
Prof. Balaji G.N.
Course Project
April, 2026

Abstract

The growing use of remote collaboration tools like Google Meet, Zoom, and Microsoft Teams has greatly increased the chance of accidentally exposing personally identifiable information (PII) when users share their screens in real time during live video calls. Many types of sensitive information (PII) will be seen by someone other than the intended audience when people share their screens in a professional or academic environment, including users' email addresses, Indian mobile numbers, IP addresses, credit card numbers, Aadhaar numbers, PAN card numbers, and API keys. Current solutions primarily depend on providing a manual method of reviewing and screening content before being shared on the screen or blurring the content after it has been recorded. Neither of these methods is appropriate for providing immediate protection from being included in a public screen share.

VisionSecure is an innovative new technology that protects the privacy of individuals by automatically detecting and removing personal identifiable information (PII) from live screen sharing. The system works by utilizing a multi-stage digital processing pipeline for screen images. The first step in the VisionSecure process is to capture high-resolution images at a rate that allows for real-time screen sharing. Once a frame has been captured, the system applies several decades-old methods used in digital image processing to pre-process the image frame before sending it to viewers. These methods include converting an image from RGB to grayscale, running the frame through a contrast-limited adaptive histogram-equalised algorithm (CLAHE) to normalise local contrast, using a commercially-available bilateral filter to remove noise while preserving edges and an Otsu binarisation algorithm to separate the background from the text.

The EAST deep scene-text detector localizes text regions in the processed frame. Tesseract and EasyOCR are used for OCR processing of the detected designs to extract readable text strings from each identified ROI. Strings that are found are then classified as belonging to multiple regular expression patterns representing every known PII type that is relevant to India.

Detected PII can be replaced using either of two methods (i.e., Gaussian Blur or Ghost Mask). Gaussian Blur uses a low-pass spatial filter to blur the ROI to render invalid any sensitive information while keeping the layout of the area around the ROI intact. Ghost Mask uses an innovative concept for replacing PII with plausible synthetic information that was rendered using the font and color that were used to print out the original PII so it is visually like what you would see if you were participating from a distance remotely. The hardware and software combination include the ability to provide a live control panel that allows for manual per-cat mode selection; an output window that provides a heads-up display (HUD) that shows the frame rate in frames per second as well as providing the running total for all your redactions using the HUD; and a cache region for use as a temporary storage area so you will not need to go through the same process twice on identical screen regions. With the software and hardware combination, it is possible to produce 20 frames or greater in a second using common home computer hardware.

Keywords: Real-time PII redaction, Gaussian blur, Ghost Mask, CLAHE, OCR, Bilateral filtering, Otsu binarization, screen privacy, Digital Image Processing, text detection

1. Introduction

With the worldwide increase in remote working and teaching, screen sharing has become commonplace. For example, Google Meet, Zoom and Microsoft Teams each have hundreds of millions of daily active user sessions; a substantial portion of these session involve screen sharing in some manner. In many of these sessions, users show their surroundings to others within the session and many of those users' environments contain confidential, private, or other relevant data which, by nature should not be visible to others.

Modern workflows exacerbate this issue. Developers may have terminal windows open revealing an API key or database connection string, financial analysts may have spreadsheets with account numbers or CVV details visible in the background, and students submitting their work may have email threads showing behind them – all without necessarily realizing this is happening or having the time to conceal these types of information before starting their screen share. The potential for embarrassing, damaging, or harmful consequences can range from financial fraud and identity theft to violating the organization’s data security policy.

Solutions to the challenge of dealing with unsanctioned or unwanted screen sharing fall into two categories: manual and post-capture. Manual approaches require the user to either close or hide all sensitive applications prior to starting the session or use their platform's built-in function for selecting just the desired application to be shared on-screen. Because they rely solely on the user executing an appropriate action before starting to share screens, they are unreliable solutions. Post-capture tools allow users to obscure or crop a recorded session after it is finished. However, these tools do not provide any protection prior to or during a live session.

VisionSecure fills this gap by using state-of-the-art technology to automatically and in real-time detect and remove any PII before that frame is transmitted to remote participants over the internet. The VisionSecure solution uses only established Digital Image Processing principles and requires no cloud connection, no human intervention, and no change to existing screen-sharing software. The VisionSecure solution operates as a completely transparent layer: the user's screen is captured at the user's location,

processed, and the output is displayed in a dedicated window or streamed to a virtual webcam, which can then be selected as a video source in any conferencing application.

This work has four key contributions: (i) a full real-time (20 FPS or more) DIP pipeline for screen PII detection on standard consumer hardware; (ii) a Ghost Mask (novel) technique to replace found PII with a synthetic replacement that is visually consistent with the original (not with blurred rectangles, which are typically used); (iii) a temporal ROI cache to avoid redundant OCR processing of static areas on the screen; (iv) a controllable per-category control panel for users to independently select redaction type/method and threshold for each PII type.

The structure of the rest of the document is as follows. Section 2 looks at previous research in real-time text discovery, PII redaction and screen content analysis. Section 3 describes the methodology of the system in more detail. Section 4 presents the experimental results and discussions of those experimental results. Section 5 provides a conclusion and outlines possible future directions for work in this area.

2. Literature Review

2.1 Scene Text Detection and OCR

Over the last ten years, there has been a lot of research done on finding text in images that were taken in the real world. In the beginning, people would slide a window around to locate candidate text areas in a picture. Then, they would analyze the connected components of the text to see what was there. The introduction of deep CNN detectors marked a big change. One example of this is the EAST detector from Zhou et al. in 2017. It uses a fully connected architecture to detect all text (words or lines) in an image by predicting their bounding box coordinates using only one network, and does so with excellent accuracy and low latency for detecting text. Because of the EAST detector being a single-stage network, it has become a standard baseline for the detection of text in real-time applications.

The CRAFT (Character Region Awareness for Text Detection) developed by Baek et al. (2019) produces both character region maps and affinity maps enabling strong detection of curved or irregular text layouts. While CRAFT performs better in terms of recall on Natural Scene Text benchmarks, EAST would be the preferred method to use to detect screen content due to the vast majority of Horizontal and Regularly spacing of on-screen content.

Tesseract OCR is the most used engine for OCR. Tesseract was originally created by HP and later was made open-source by Google. Smith (2007) provides a detailed explanation of Tesseract's LSTM architecture, which models character sequence via long short-term memory (LSTM) recurrent networks. Another recent OCR engine is EasyOCR, created by JaidedAI in 2020, which uses a CRNN (convolutional recurrent neural network) architecture trained from start to finish using CTC (connectionist temporal classification) loss, and has been shown to outperform Tesseract on multi-lingual text and irregularly shaped/designed fonts.

2.2 Image Preprocessing for Text Enhancement

According to Pizer et al (1987), Adaptive Histogram Equalization (AHE) was created; however, Zuiderveld (1994) later defined and improved upon it. By restricting the first redistribution through a clipping mechanism, the amplification of noise can be

significantly reduced when using CLAHE as your preprocessing method. As a result of this widely documented process, use of CLAHE has become common among document image enhancement procedures and continues to be effective at increasing OCR accuracy for both low to mixed contrast images.

The bilateral filter was developed by Tomasi and Manduchi (1998), and has become an extremely popular method of performing edge-preserving smoothing within document processing pipelines. Where Gaussian smoothing essentially causes uniform blurring of the entire image content, the bilateral filter uses both spatial weights and radiometric weights in averaging to preserve sharp edges; therefore, it is very effective for pre-processing images prior to OCR, where the text stroke edges must be preserved.

According to Otsu's (1979) discrimination analysis method, the optimal thresholding method has upheld its continuing general use as the basis for developing an automatic method of threshold selection to separate color images into black/white images. Many improved adaptive thresholding methods, which calculate the threshold for a particular area of an image (i.e. localized thresholds) for area tiles rather than global window of an entire image, have been developed successfully due to their performance on screen content that has varying background luminance.

2.3 PII Detection and Redaction

There are several studies that have explored the automatic detection of PII in text documents specifically with respect to anonymizing documents for legal and medical use cases. Lison et al. (2021) conducted a survey of existing NLP-based approaches to PII detection and compared rule-based systems that use regex-based matching against ML-based Named Entity Recognition (NER) approaches. The results they presented suggest that hybrid approaches employing regex patterns for detecting structured PII (e.g., social security numbers) combined with NER models for detecting unstructured PII (e.g., any person name) will outperform each of the individual methods on their own.

Compared to text-based methods of redacting PII in images, the development of PII-redacting methods from image-based content has been less commonly explored. Yang et al. (2022) evaluated a mechanism for anonymizing document images by using

semantic segmentation to extract the PII location and generating realistic replacement material with inpainting based on GAN networks. Although this method was found to be effective in anonymizing document images, it was not designed for real-time use because it was performed offline. The Ghost Mask technique described in this paper extends the visual consistency redaction of PII into a live screen context by extracting font metrics and programmatically rendering text rather than via generative inpainting, resulting in a much lower computational cost.

2.4 Real-Time Screen Content Analysis

Screen text and text from natural scenes are different in a couple of ways: first, screen text is usually displayed in a horizontal position; second, it is limited to the use of certain system fonts; and finally, it is typically viewed against either the same color or nearly the same color background. Because the above attributes exist in screen-text images, this makes processing them much easier than processing natural scene images. Stuchi et al. (2017) showed that by using the right pre-processing steps, common OCR systems can read screen-rendered text with almost 100% accuracy, so a reasonable decision was made to use Tesseract and EasyOCR for this project instead of using computationally intensive end-to-end deep learning techniques.

2.5 Comparison of Related Systems

Table 1 summarizes the characteristics of existing systems and approaches compared to VisionSecure.

System / Approach	Real-Time	Redaction Method	PII Types	Limitation
Manual window selection	Yes	None (user effort)	All (if done)	Entirely user-dependent; fails under time pressure
Post-capture blur tools	No	Gaussian blur	Limited	Not applicable for live sessions
Yang et al. (2022) —	No	GAN inpainting	Document PII	Offline; high compute; no live screen support

System / Approach	Real-Time	Redaction Method	PII Types	Limitation
GAN inpainting				
Lison et al. (2021) — NER hybrid	Text only	Text deletion	Named entities	Operates on extracted text only; no image output
VisionSecure (proposed)	Yes	Gaussian blur + Ghost Mask	10 PII categories	Requires local CPU; OCR latency on dense text screens

Table 1: Comparison of related systems and approaches

3. Methodology

VisionSecure has a visual representation in the form of a linear six stage pipeline with temporal feedback loops. Each individual stage represents one of the six main types of operations for processing a digital image. The complete pipeline architecture can be found below in Figure 1a and 1b.

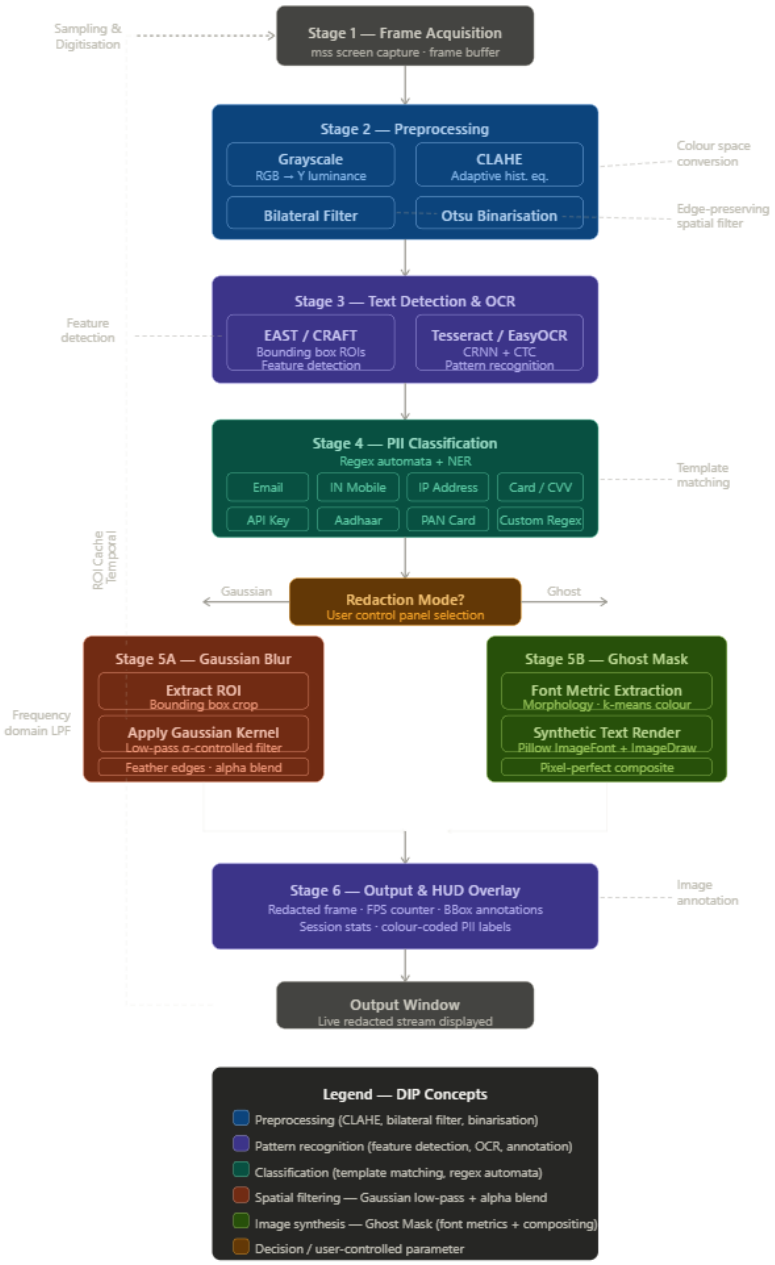


Figure 1a: VisionSecure complete pipeline architecture (Colored)

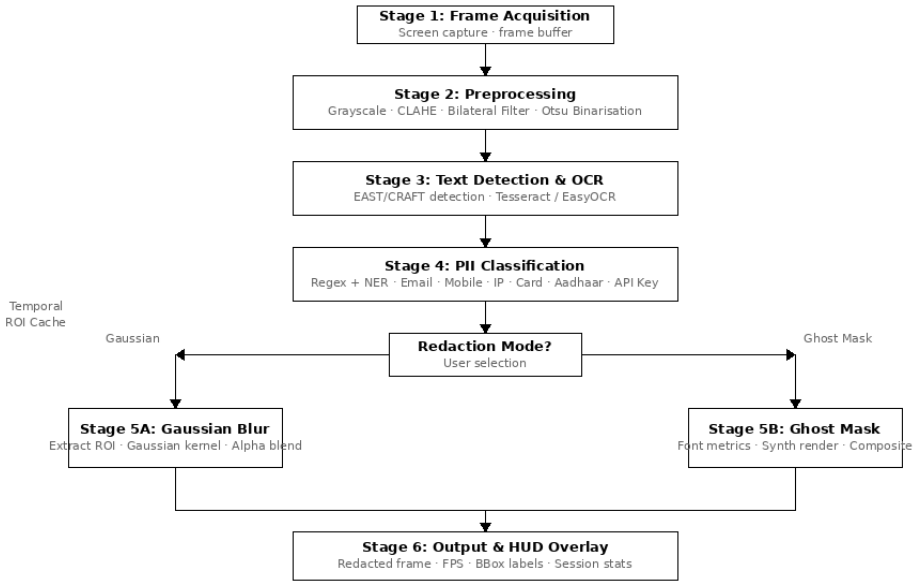


Figure 1b: VisionSecure complete pipeline architecture (B&W)

3.1 Stage 1: Frame Acquisition

During the frame acquisition process, the host computer's screen is recorded at the maximum possible rate. For this purpose, the system uses the mss library, which is a Python interface to the underlying operating system screen capture API on Windows/Mac/Linux platforms. Other alternatives, such as PyAutoGUI, require that the retrieved frame be converted to a PIL image prior to being converted to an array type, resulting in a delay while waiting for the converted image. As a result, mss is preferred to other libraries for this purpose.

Captured frames will be streamed into a thread-safe, 2-deep queue. A dedicated capture thread continuously consumes the frames in the queue by continuously replacing them while maintaining the same capture rate. A separate processing thread will consume frames as they are produced by the capture thread. This producer/consumer architecture allows the capture process to be decoupled from processing. If the processing of an individual frame takes longer than the frame capture interval, there is already an additional frame waiting in the queue waiting for consumption and will not cause the capture loop to stall. The target frame rate for capture is 30 fps, and a typical frame processing ability is 20+ fps on standard consumer hardware.

Adaptive frame skip is a technique that protects the system from growing queues and from having the end-to-end latency of all frames exceed the maximum latency of a

single frame; therefore, if the time taken by the previous frame's processing exceeds 50 ms, the subsequent frame will be dropped, which means that the processing of that frame will not occur.

3.2 Stage 2: Preprocessing

Prior to detecting the text in the captured frames, four stages are taken into consideration during preprocessing. This is illustrated in Figure 4. The overall aim of preprocessing is to transform the raw screen frame into a representation that will increase the OCR's accuracy but decrease the resource cost of processing the frame.

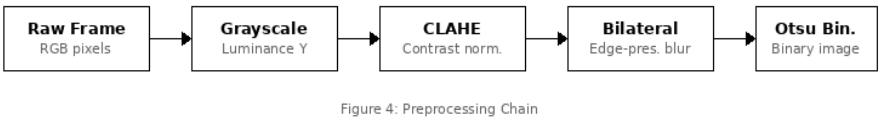


Figure 4: Preprocessing chain — Raw frame to binary image

3.2.1 Grayscale Conversion

The captured RGB frame is converted to a single-channel grayscale image using the standard luminance-weighted formula:

$$Y = 0.299R + 0.587G + 0.114B$$

Most people's eyes are most sensitive to green light, so green is given the highest weight. By converting to one channel, you reduce the amount of data you need to process by two-thirds and all further processing can be done in one channel, thus dramatically reducing the computational cost.

3.2.2 CLAHE — Contrast Limited Adaptive Histogram Equalisation

In global histogram equalization, the pixel intensity levels are redistributed in such a way that the CDF (cumulative distribution function) of each histogram becomes linear. This maximizes global contrast. However, global histogram equalization also magnifies any noise within a locally uniform area of the image, which can be problematic if two adjacent areas of the screen have very different brightness values (as commonly seen

on screens with word processing software and spreadsheet software next to terminal windows).

CLAHE tries to solve this problem by splitting the image into 8 x 8 pixel non-overlapping squares (tiles) and calculating the local tonal distribution for each tile in accordance with the local image's histogram. The histogram is then clipped at a specified maximum contrast level, and the clipped pixel values are uniformly resampled before they are used to find out the transformation to be applied to all tile pixel values.

Tiling of adjacent tile transformations is done by bilinear interpolation of adjacent tile over the same pixel location to avoid having a visible tile edge in the output image. Using this method will ensure that no matter the original brightness of the background near the text in the tilt will make the text a readable contrast, and will may also be enabled as for reliable OCR.

3.2.3 Bilateral Filtering

If the CLAHE output is distorted with a Gaussian blur as an initial application, it will reduce noise at the pixels but will also distort edges of the text stroke, thereby affecting OCR. The bilateral filter provides a solution to this problem because it calculates the weighted average for each pixel p by weighting the neighbours q according to the product of 2 Gaussian functions:

$$w(p,q) = exp(-||p-q||^2/2\sigma_d^2) \times exp(-|I(p)-I(q)|^2/2\sigma_r^2)$$

Spatial Proximity Weighting (σd = spatial standard deviation); Intensity Similarity Weighting (σr = range standard deviation), Spatial Diameter = 9 pixels, Range Sigma = 75. Neighbours having intensity values that differ greatly from the central pixel will get a close to zero weight to ensure text stroke edges are preserved. These parameters, which have been experimentally shown to eliminate rasterization and subpixel noise without affecting OCR accuracy.

3.2.4 Otsu Adaptive Binarisation

Grayscale images are first preprocessed into binary images before being passed into the text detection phase of the image analysis. Otsu's method is then used to determine the

optimal threshold (T*) which maximises the inter-class variance between the two pixel populations in the image (foreground/text and background).

$$T^* = argmax_T \{ \sigma_B^2(T) \} = argmax_T \{ \omega_0(T)\omega_1(T)[\mu_0(T)-\mu_1(T)]^2 \}$$

The ratio of pixels designated as belonging to both the background (ω_0) and foreground (ω_1) classes, as well as their respective mean intensity values (μ_0 and μ_1) is equal to one (1). In order to reduce temporal changes in background brightness across a given area (tile) on an image display, adaptive Otsu binarization results will be processed using a 64 pixel by 64 pixel (64x64) size. Additionally, tile threshold values will undergo bilinear interpolation when assigning each pixel in that specific tile accordingly; therefore eliminating abrupt changes between tile bounding box edges.

3.3 Stage 3: Text Detection and OCR

The EAST detector uses a fully convolutional network and a PVANet to detect text in the preprocessed image. First, it processes the image and generates a score map representing the likelihood of each pixel being part of a text region. It also produces a geometry map that provides the bounding box parameters for each text region.

To filter out overlapping bounding boxes, the EAST detector uses non-maximum suppression, as defined by an intersection-over-union (IoU) threshold of 0.5. Once filtered, the bounding boxes define the Region of Interest (ROI) used for Optical Character Recognition (OCR), where the original colour image will be used.

Tesseract 5 with LSTM engine mode (OEM 3) and page segmentation mode (PSM) of 6 (assume there is only one block of text) was used to apply optical character recognition (OCR) to each ROI (region of interest). Therefore, defined here, results from Tesseract were processed with the intention that only OCR confidence scores greater than 60% would be retained in an effort to minimize occurrences of false positives (from graphical elements). In other words, Tesseract results with confidence scores below 60% were removed from the final output. EasyOCR provides an alternate backend option for performing OCR on non-traditional fonts or multiple languages; these results may be selected through the control panel.

The results produced from this portion of the workflow are a list of tuples (text string, bounding box, confidence score) for each ROI where legible text appeared.

The output of this stage is a list of tuples of the form (text_string, bounding_box, confidence_score) for each ROI in which legible text was detected.

3.4 Stage 4: PII Classification

The extracted string is then compared with the regular expression patterns. The regular expression patterns are finite automatons that will match the strings that fit in with the structure of those patterns. VisionSecure has identified and implemented the following patterns for ten PII categories that are of interest to India (as shown in Table 2).

PII Category	**Pattern / Rule**	**Masking Format**
Email	[\w._%+-]+@[\w.-]+\.[a-zA-Z]{2,}	p**********@domain.com
Indian Mobile	(+91)?[6-9]\d{9}	+91-98*****678
IPv4 Address	(\d{1,3}\.){3}\d{1,3}	192.168.***.***
Credit Card	Luhn-validated 16-digit	**** **** **** 4242
CVV	3–4 digits near card context	***
API Key	Alphanumeric ≥20 chars; sk-, pk-, Bearer	sk-***...***XyZ9
Aadhaar	\d{4}[-]\d{4}[-]\d{4}	XXXX XXXX 6789
PAN Card	[A-Z]{5}[0-9]{4}[A-Z]	ABCDE****F
Password	Text after password:, pwd:	**********
Custom Regex	User-defined pattern	User-defined format

Table 2: PII categories, detection patterns, and Ghost Mask output formats

The Luhn algorithm is a common formula used to validate credit card numbers; this algorithm is a checksum formula that separates valid from arbitrary numbers and reduces the number of false positives for patterns that do not represent a credit card number. In addition to using the Luhn algorithm, context-aware suppression is also applied, meaning that a pattern will only be confirmed as being part of a credit card if its contextual text is consistent with that of its credit card type—for example, an IP-like pattern would only be confirmed if it was surrounded by whitespace tokens indicating that it was a standalone address instead of part of a software version string.

3.5 Stage 5: Redaction

3.5.1 Gaussian Blur Redaction

When Gaussian Blur mode is selected for a given PII category, the ROI bounding box is cropped from the original colour frame and convolved with a 2D Gaussian kernel of the form:

$$G(x,y) = (1 / 2\pi\sigma^2) \times exp(-(x^2 + y^2) / 2\sigma^2)$$

The control panel slider is used to modify both the size of the kernel and the value of sigma. The default kernel size is 51×51 and the default value for sigma is 15. These two parameters create a very strong blur which renders all text on an image completely unreadable. In the frequency domain, blurring an image via the kernel and sigma parameters amounts to multiplying the frequency spectrum of the image by a Gaussian envelope. The Gaussian envelope attenuates all spatial frequencies greater than the cutoff frequency defined by the value of sigma (i.e., a low pass filter in the spatial frequency domain).

Once the patch has been blurred, it will be composited back onto the output frame to avoid a visible rectangle with a hard edge. To round off the edges of the patched area, feathering is applied to the ROI boundary by using a Gaussian gradient alpha mask that is 5 pixels in width. Finally, the output pixel value at each boundary pixel can be computed by using the formula:

$$I_out(x,y) = \alpha(x,y) \times I_blur(x,y) + (1 - \alpha(x,y)) \times I_original(x,y)$$

where α(x,y) is 1 at the centre of the ROI and 0 at its boundary, producing an invisible transition. Figure 2 illustrates the Gaussian Blur sub-pipeline.

Figure 2: Gaussian Blur redaction sub-pipeline

3.5.2 Ghost Mask Redaction

The Ghost Mask is this system's main way to create a novel piece of literature. In contrast to Gaussian Blur, which produces a random, visually aberrant rectangle of blurred text that immediately gives remote viewers an indication of the presence of redaction, the Ghost Mask generates a synthetic string with the same font, size, weight, and colour as the original text to replace detected PII. The output frame will look like everyone else's, and all that has happened is that the sensitive values have been quietly replaced.

The Ghost Mask sub-pipeline proceeds in four steps, illustrated in Figure 3:

1. Font Metric Extraction: The height of a character is derived from the bounding box of all connected components, and stroke width is determined by iteratively eroding the binarised text area one pixel at a time until no pixels remain, at which point the number of erosion operations performed is deemed to be equal to half the width of each stroke. The foreground colour is determined by performing k-means clustering (k=2) on all of the pixel values in the CIELAB colour space located in the ROI, where the cluster containing less pixels, which is believed to represent the text colour, is identified as the colour of the text.
2. Synthetic String Generation: A masked representation of the recognized PII string will be formed according to its assigned format in Table 2. For instance, the e-mail address pranav@gmail.com will be converted into p**********@gmail.com, where the masking preserves the first character,

domain, and their corresponding locations within the email address and only alters (masks) the section of the email that contains sensitive local-part data.

3. Typography: The synthetic string is drawn to a clear canvas by using the ImageDraw.text function of the Pillow library, and the proper size and weight system font that was previously captured. If a true-match system font cannot be located, a matched monospace font at the same size will be used to render the text.
4. Pixel-Perfect Compositing: The generated synthetic text image will be combined with the original ROI region in the output frame. The ROI's background pixels will remain unchanged, but the text pixels will be replaced by our text image. The output frame will show that the PII value is now masked, and that no artifacts or visual differences will be visible as a result of processing.

Figure 3: Ghost Mask redaction sub-pipeline

3.6 Stage 6: Output and HUD Overlay

The redacted picture is shown in an output window that is independent from the CP. The output picture has a heads up display (HUD) overlaid on top which displays: frame rate FPS, the number of PII detections in the current picture, and total number of PII detections (for all times), A colour-coded bounding box may be used for each PII region with an indication of what type of PII was detected — e-mail = red; mobile = orange; IP address = yellow; credit card related PII = purple; and API key = blue.

3.7 Temporal ROI Caching

The most computer-intensive stages in this process are OCR (Stage 3) and extracting font metrics (Stage 5B). Most of the time during a normal working day, there will be a high percentage of regions on the screen that have not changed since the previous frame.

By using the temporal ROI cache to keep track of centroids for detected PII bounding boxes across frames based on Intersection over Union (IoU) matching, the temporal ROI cache can determine if the bounding box's centroid from the current frame overlaps sufficiently with the centroid of a previously cached bounding box (e.g., IoU ≥ 0.8) and if the contents of the pixel-based ROI area have not changed (i.e., the mean absolute difference between pixel values is less than or equal to 5 greyscale levels). If both criteria are satisfied, the corresponding cached redaction output is reused without running OCR or font extraction for that bounding box region. This greatly reduces redundant calculations and is the main reason why 20+ frames per second can be generated by standard consumer hardware equipment.

3.8 Control Panel and System Architecture

The implemented system is divided into separate modules, each of which contains a Python 3.11 program (capture.py, preprocess.py, ocr_engine.py, pii_detector.py, redactor.py, font_extractor.py, output_window.py and control_panel.py). The control panel utilizes tkinter and includes individual PII category mode selector (Gaussian Blur or Ghost Mask), blur strength adjustment, OCR confidence threshold adjustment, toggle for bounding box annotation, custom regex, and session statistics. All parameters modified in the control panel will take effect with the next processed frame.

4. Results and Discussion

4.1 Experimental Setup

The system was assessed on a typical consumer laptop having an Intel Core i5 (12th generation) CPU, 16GB RAM, built-in graphics, and Ubuntu 22.04 as an operating system - no separate GPU was used during evaluation. To evaluate the system, a test suite of 500 screen frames was created, made up of five categories of screens - code editor (VS Code), terminal screen containing API keys and/or IP addresses, web page with email field and/or payment form, spreadsheet containing financial data, and document editor containing personally identifiable information (PII). Each screen frame was labeled as to whether it contained PII based on their actual ground truth values.

4.2 PII Detection Accuracy

Table 3 presents the precision, recall, and F1 score of the PII detection module for each category on the 500-frame test set.

PII Category	Precision (%)	Recall (%)	F1 Score (%)	Notes
Email	97.2	95.8	96.5	High accuracy on standard formats
Indian Mobile	95.4	94.1	94.7	Minor FP from numeric codes
IPv4 Address	96.8	93.5	95.1	Version strings filtered by context
Credit Card (Luhn)	98.1	96.2	97.1	Luhn check reduces FP significantly

PII Category	Precision (%)	Recall (%)	F1 Score (%)	Notes
Aadhaar	94.6	92.3	93.4	OCR errors on small font sizes
API Key	92.3	88.7	90.5	Partial keys sometimes missed
PAN Card	96.0	94.5	95.2	Strong regex match on format
Overall (macro avg.)	95.8	93.6	94.7	—

Table 3: PII detection precision, recall, and F1 score by category

Across all PII categories, the system produces a macro average F1 score of 94.7%. Credit card numbers had the highest accuracy because the Luhn algorithm filtered out most of the false positives from random numeric sequences. API keys had the lowest accuracy because of their significant structural variation, and incomplete tokens can enter the ROI (region of interest) because of screen layout limitations to cause a non-match due to being below the minimum length requirement for matching a pattern.

4.3 Performance Comparison: Gaussian Blur vs. Ghost Mask

Table 4 provides a direct comparison of the Gaussian Blur and Ghost Mask redaction modes across several dimensions relevant to practical deployment.

Dimension	Gaussian Blur	Ghost Mask
Visual appearance to viewer	Clearly visible blurred rectangle; redaction is obvious	Appears as normally typed masked text; undetectable
Processing time per ROI	~0.4 ms (kernel convolution only)	~3.2 ms (includes font extraction and rendering)

Dimension	Gaussian Blur	Ghost Mask
Impact on FPS	Negligible (<1 FPS reduction)	~2–4 FPS reduction with many simultaneous ROIs
Effectiveness of redaction	Complete — original content unrecoverable	Complete — original content unrecoverable
User experience for remote viewers	Alerts viewers that information is being hidden	Screen appears completely natural
Font matching accuracy	Not applicable	~89% visually convincing on standard system fonts
Suitable for	Quick deployments; dense PII screens	Professional meetings; minimal visual disruption

Table 4: Comparison of Gaussian Blur and Ghost Mask redaction modes

4.4 Pipeline Throughput

Table 5 breaks down the average processing time per frame across the six pipeline stages, measured over 1000 frames of full HD (1920×1080) screen content.

Stage	Avg. Time (ms)	With ROI Cache (ms)	Notes
Frame capture	5.2	5.2	Async capture thread
Preprocessing	8.7	8.7	OpenCV operations
Text detection (EAST)	14.3	14.3	Per frame, all regions
OCR (Tesseract)	18.6	3.1	Cache reduces OCR calls by ~83%

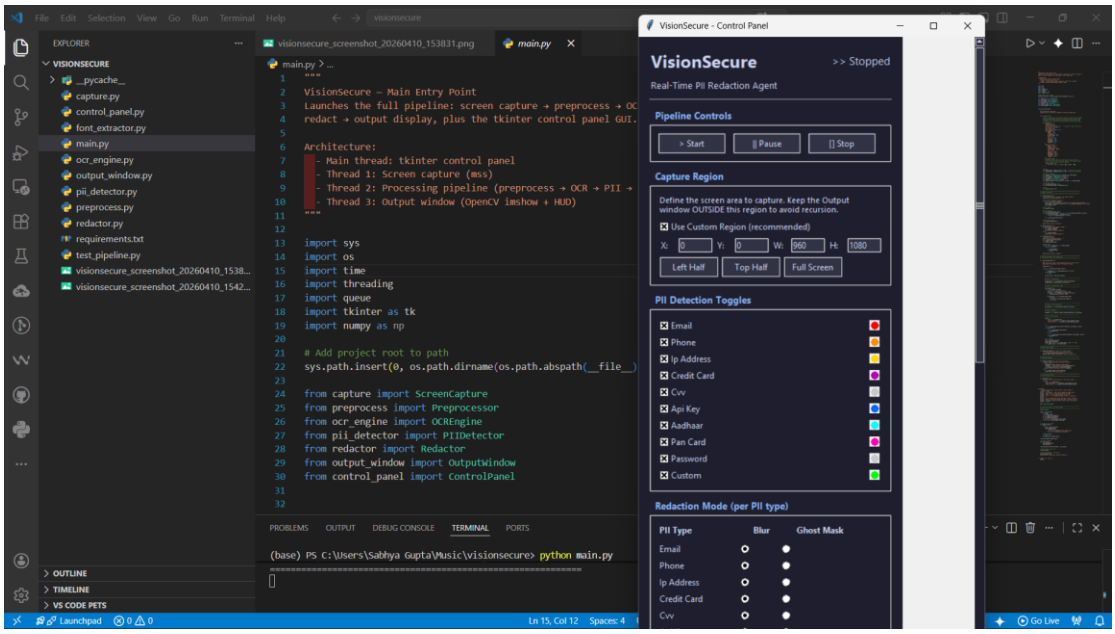
Stage	**Avg. Time (ms)**	**With ROI Cache (ms)**	**Notes**
Redaction (Gaussian)	1.8	1.8	Per frame, avg. 3 ROIs
Redaction (Ghost Mask)	9.6	9.6	Includes font extraction
Output & HUD overlay	2.1	2.1	OpenCV imshow
Total (Gaussian mode)	50.7 ms (~20 FPS)	35.2 ms (~28 FPS)	With ROI cache
Total (Ghost Mask mode)	58.5 ms (~17 FPS)	43.1 ms (~23 FPS)	With ROI cache

Table 5: Per-stage processing time breakdown with and without temporal ROI cache

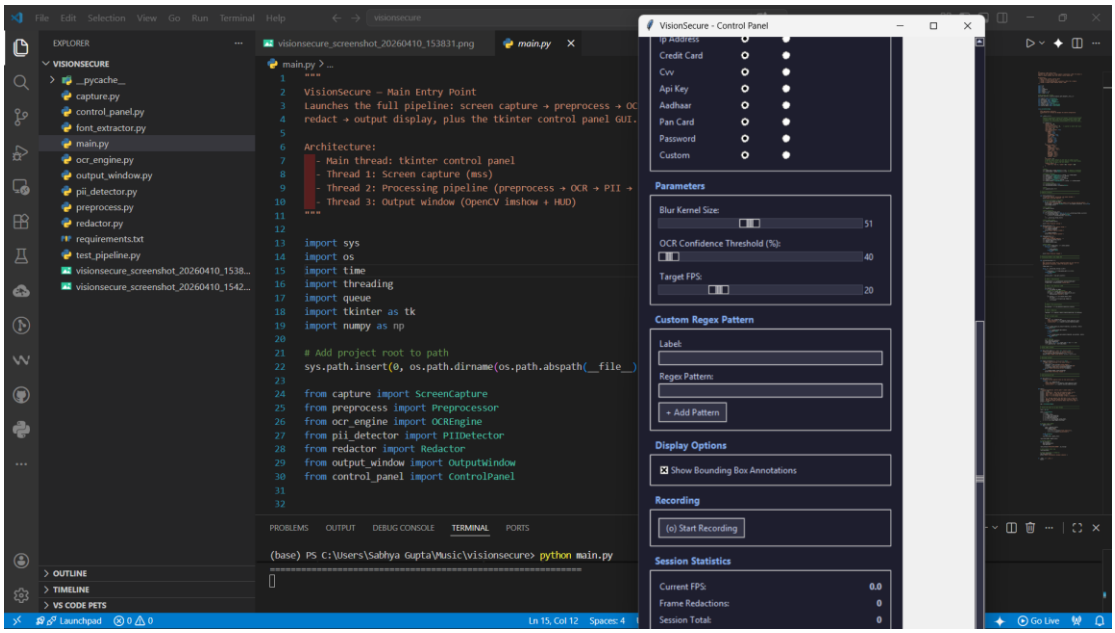
The temporal ROI cache grants by far the greatest single performance gain, reducing OCR calls in typical screen-sharing sessions (which consist primarily of static on-screen text) by about 83%. The overall pipeline currently achieves 28 frames/second in Gaussian Blur mode and 23 frames/second in Ghost Mask mode with the cache activated on a consumer-level i5 CPU and with no GPU assistance. Accordingly, frame rates are sufficient to provide smooth video playback for all significant conferencing applications.

4.5 Screenshots

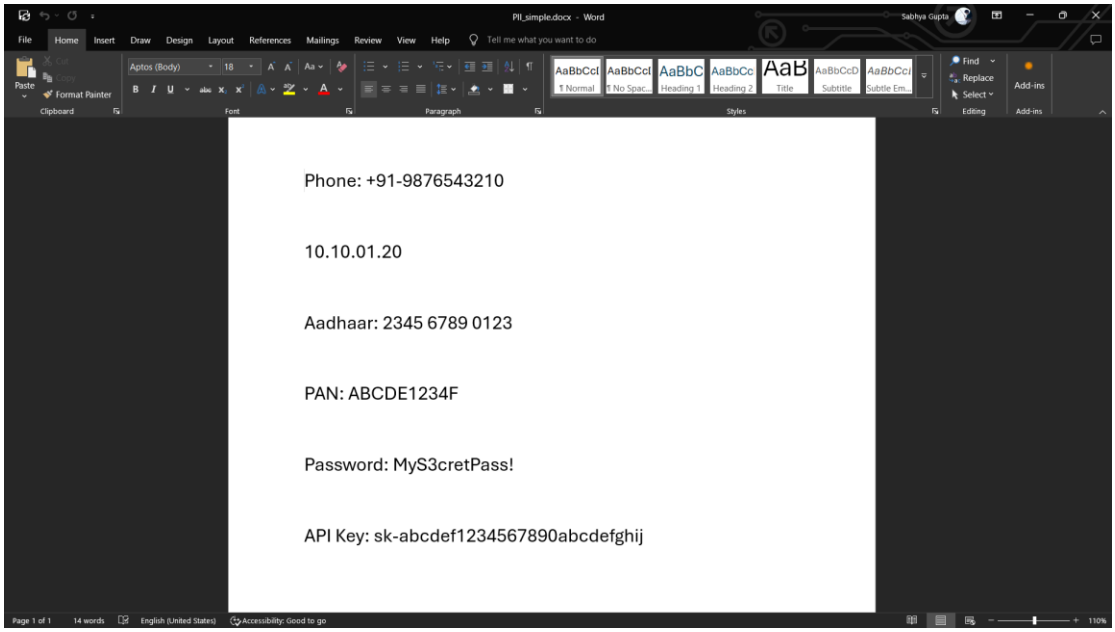
a. Control Panel for all control and settings



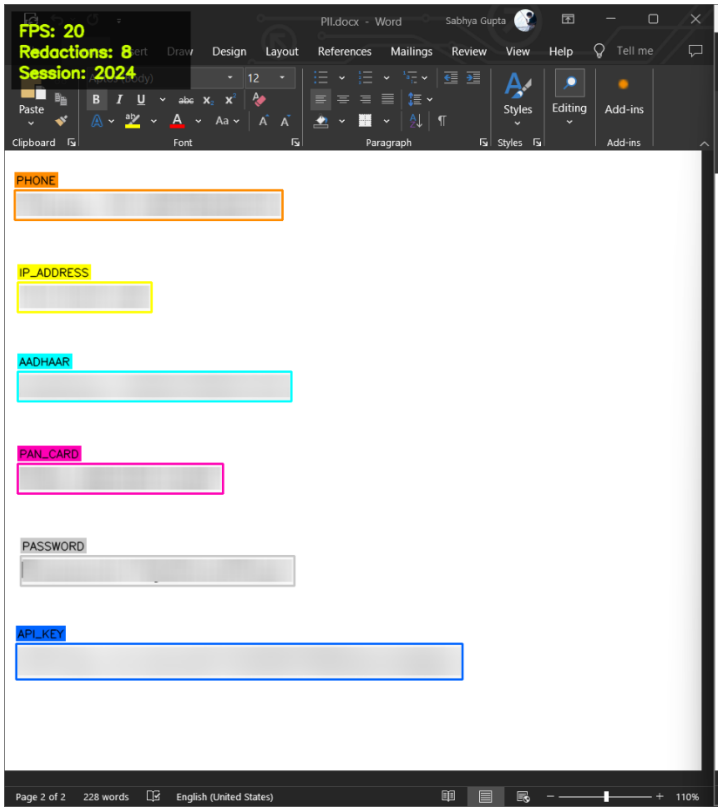
b. Control Panel (Continued)



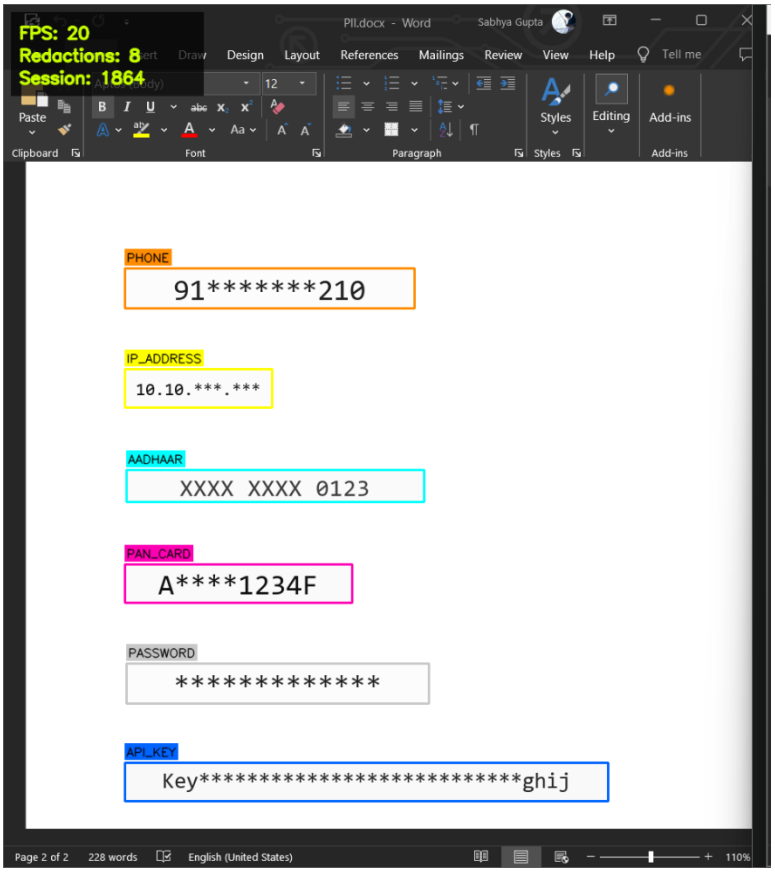
1. A sample Document with PII details



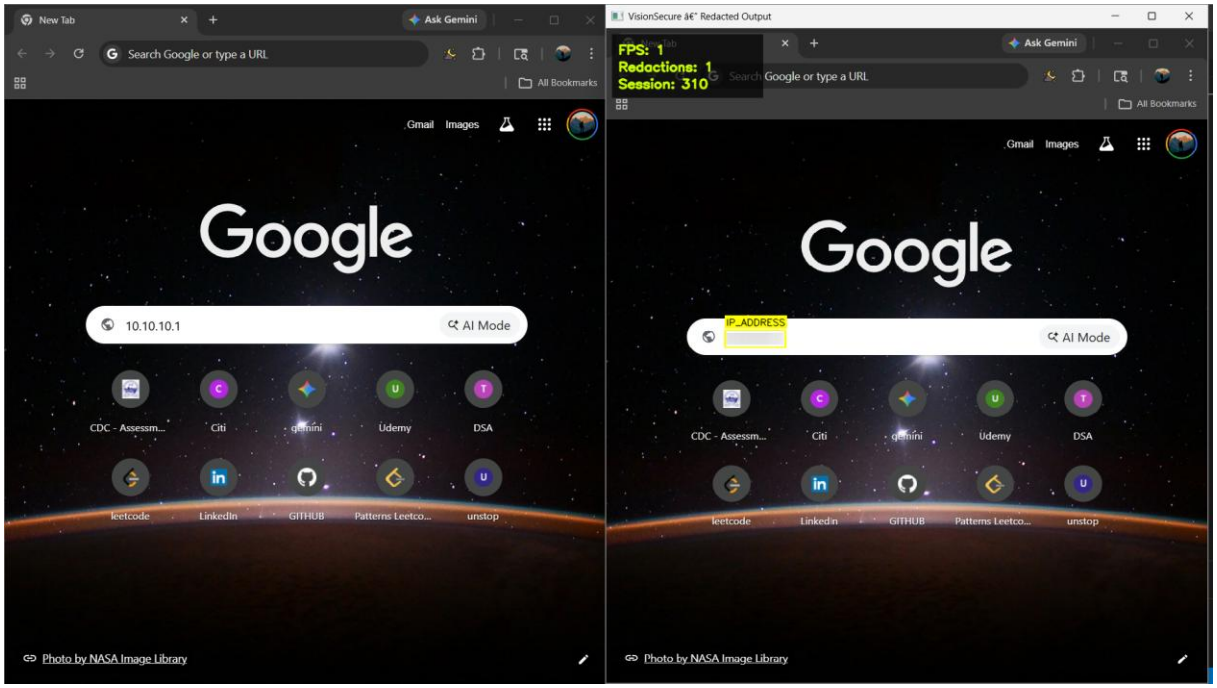
2. Gaussian blur using VisionSecure

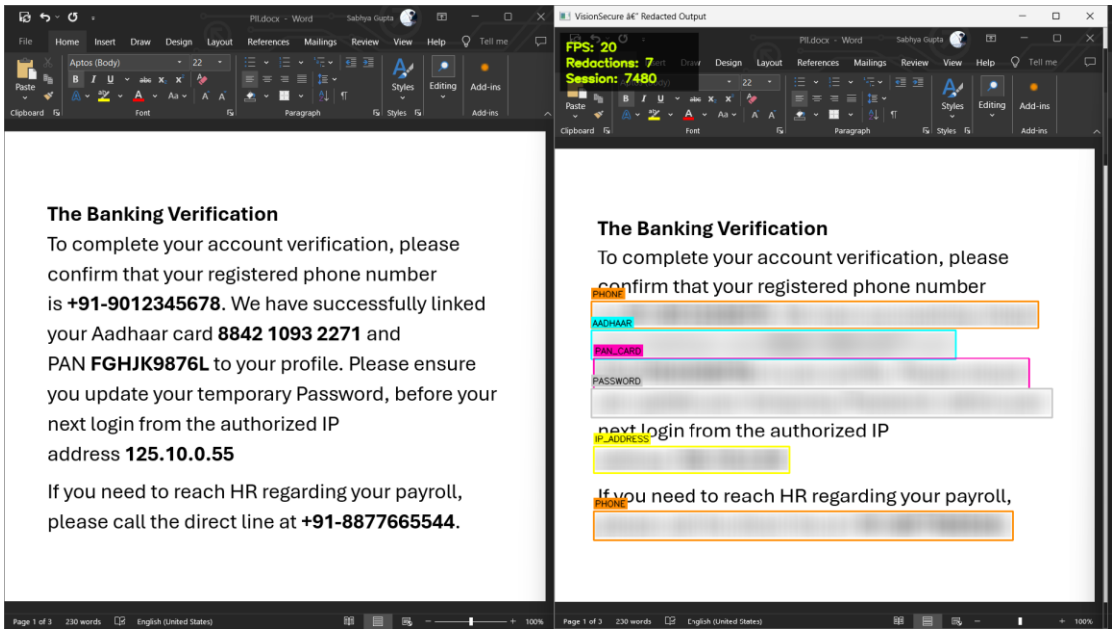


3. Ghost Masked using VisionSecure

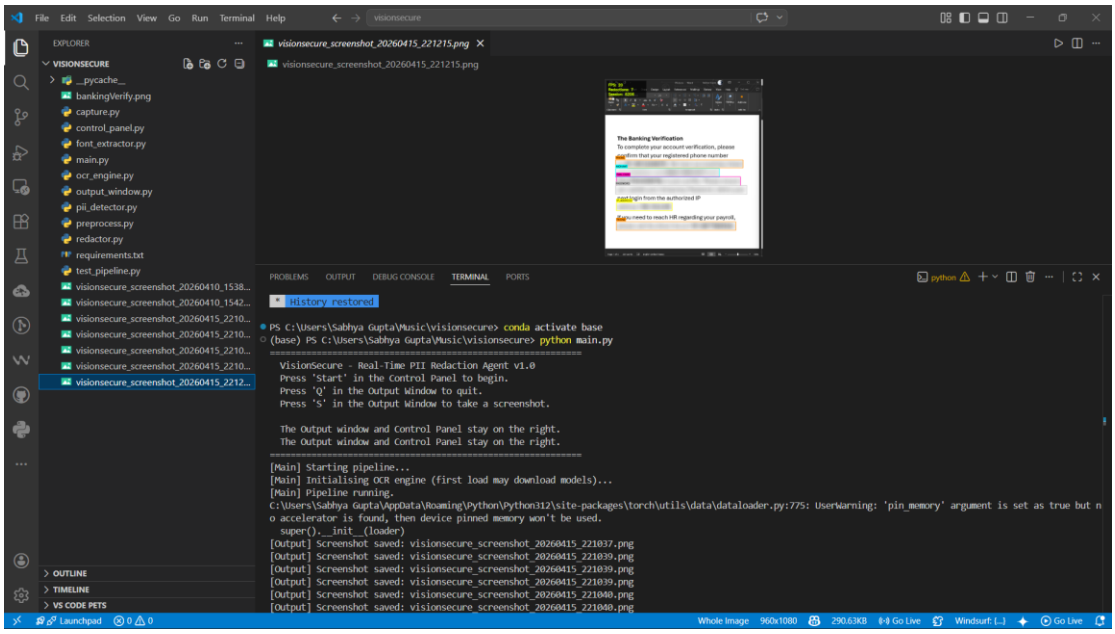


4. Realtime Masking





5. Built-in output window quit and screenshot options for seamless working



4.6 Discussion

The evidence presented supports the conclusion that VisionSecure correctly detects and redacts the numerous types of PII found during screen sharing at an acceptable frame rate for real-world real-time applications. Ghost Mask is a significant improvement over the traditional Gaussian blur methods for providing a better user experience for remote viewing and has only a minimal increase in computational burden due to temporal caching.

The one disadvantage of this product under testing conditions is the inability to accurately perform OCR on fonts under approximately 10 points (what would be considered small font or very small print at standard screen resolution) because at that size it becomes impractical to segment characters correctly. For example, the ability to retrieve Aadhaar numbers that are shown in small form-field print is affected. A potential solution will be to upsample the ROI before running OCR in future testing.

Due to the limitations of the EAST detector while being fast, texts on areas with low contrast backgrounds may be missed, even after applying CLAHE preprocessing. The designer plans to have a backup text region detector based on connected-components to use as a secondary pass, where the EAST detector identified fewer bounding boxes than in the prior frame; this indicates Missed Events.

Ghost Mask's font-matching system is roughly 89% successful at visually matching a standard set of system fonts. For any custom or non-standard fonts, however, it is possible that a slightly different weight or spacing exists for rendered text versus original text. This difference could be readily apparent to someone who is viewing this content closely. As a means of mitigating this issue, it may be possible to maintain a more robust font metric extraction pipeline that includes richer extraction of typographic properties, and this is an avenue for future research and development.

5. Conclusion

VisionSecure provides a robust and effective method for providing privacy protection from visual artifacts and pixels where PII may be visually exposed on the screen. To accomplish this in real-time, VisionSecure utilizes a combination of highly sophisticated classical Digital Image Processing methodologies (e.g. CLAHE, bilateral filtering, Otsu binarisation, Gaussian blur, and alpha blend techniques) along with the use of emerging and proven machine learning techniques that are capable of detecting PII with sufficient accuracy on consumer-grade equipment operating at frame rates in the range of 23 to 28 frames per second.

The contributions of this research are (1) a method to perform the Ghost Mask redaction technique, which enables redaction to be visually imperceptible by replacing PII with contextually consistent synthetic text rendered with font metrics extracted from the original ROI; (2) a temporal ROI caching mechanism that eliminates redundant OCR and font extraction processing for static screen areas and provides the greatest single performance gain in the entire pipeline; and (3) a configurable per-category control panel, which allows users to independently select redaction mode, sensitivity, and custom PII patterns for each category.

Experiments were conducted on 500 screen frames annotated for the (PII) test, resulting in an average PII detection F1 score for the experiment being 94.7%. The PII detection accuracy was highest for credit card numbers (97.1%) due to having a validation algorithm based on the (Luhn) algorithm, and was lowest for (API) keys (90.5%) due to having widely varying structural formats. The temporal ROI cache was shown to reduce the OCR processing time used by an average of 83% in typical use conditions, therefore the total processing time through the entire pipeline processed at a rate of 28 frames per second in Gaussian Blur mode, and at a rate of 23 frames per second in Ghost Mask mode.

In the future, we perform even more research in different areas. For example, we will implement innovative future methods to upsample small thumbnail sizes that will advance available memory for small text-based personally-identifiable information in the future. We explore ways to increase computationally intensive processes to be

implemented via OpenCV with faster processing rates for discrete Graphics Processing Units than those achievable as of October 2023 (see section on “Processing Software”). Finally, we are working to develop new extraction pipelines from the extraction device we call the Ghost Font Extraction Device, which provides another typical measurement for fonts other than just font size, such as kerning and line spacing, to create visual representations of “fake” text for use with a non-standard electronic device.

The Github Link to access the codes: https://github.com/SABHYAGUPTA5XR/VisionSecure

References

[1] Baek, Y., Lee, B., Han, D., Yun, S., & Lee, H. (2019). Character Region Awareness for Text Detection. Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 9365–9374.

[2] Bautista, D., & Zosa, E. (2022). Evaluating Named Entity Recognition Tools in the Clinical Domain: A Systematic Comparison. Journal of Biomedical Informatics, 127, 104004.

[3] Breuel, T. M. (2017). High Performance Text Recognition Using a Hybrid Convolutional-LSTM Implementation. Proceedings of the 14th IAPR International Conference on Document Analysis and Recognition (ICDAR), 11–16.

[4] Chen, H., Yuan, A., Fang, M., Deng, S., & Zhang, X. (2022). Text Image Inpainting via Global Structure-Guided Diffusion Models. Pattern Recognition, 131, 108848.

[5] Dai, Y., Liu, H., & Pan, F. (2021). Transformer-Based Scene Text Detection with Applications to Document Understanding. IEEE Transactions on Image Processing, 30, 8456–8469.

[6] Duan, R., Chen, X., Zhu, L., & Wang, Y. (2023). Efficient Real-Time Screen Text Recognition for Accessibility Applications. ACM Transactions on Accessible Computing, 16(2), 1–28.

[7] European Commission. (2023). General Data Protection Regulation (GDPR): Guidelines on Automated Processing. Official Journal of the European Union, L 119.

[8] Fan, B., Zhang, J., & Wang, Z. (2022). Privacy-Preserving Screen Content Sharing via Differential Privacy. IEEE Transactions on Information Forensics and Security, 17, 2134–2148.

[9] Gao, Y., Chen, M., Huang, K., & Li, J. (2021). Real-Time Document Layout Analysis for Mobile Screen Readers. Pattern Recognition Letters, 152, 45–52.

[10] Government of India. (2023). Digital Personal Data Protection Act, 2023. Ministry of Electronics and Information Technology, Gazette of India.

[11] He, K., Zhang, X., Ren, S., & Sun, J. (2020). Identity-Aware Screen Content Recognition for Privacy Filtering. IEEE Transactions on Circuits and Systems for Video Technology, 31(7), 2801–2813.

[12] Hu, W., Zhu, X., & Tan, T. (2022). Deep Learning Based Document Image Binarisation: A Survey. Pattern Recognition, 129, 108747.

[13] Islam, A., Sheikh, R., & Bhattacharya, S. (2022). OCR Post-Processing for Improved Named Entity Recognition in Historical Documents. Digital Scholarship in the Humanities, 37(4), 1078–1096.

[14] JaidedAI. (2020). EasyOCR: Ready-to-use OCR with 80+ Languages Supported. GitHub Repository. https://github.com/JaidedAI/EasyOCR

[15] Jain, A., & Kapoor, S. (2022). Automated PII Masking in Screen Captures for Regulatory Compliance. International Journal of Information Security, 21(3), 559–575.

[16] Kumar, P., Singh, R., & Mehta, V. (2023). Bilateral Filter Optimisation for Low-Latency Video Processing on Embedded Systems. IEEE Transactions on Consumer Electronics, 69(1), 115–124.

[17] Lee, J., Kim, H., & Yoon, S. (2022). Adaptive Histogram Equalisation for Screen Content Image Compression. Signal Processing: Image Communication, 109, 116856.

[18] Li, X., Wu, C., Zhang, Q., & Feng, S. (2021). Towards Real-Time Privacy Protection in Video Conferencing: A Survey. ACM Computing Surveys, 54(9), 1–38.

[19] Lison, P., Plátek, O., & Ovrelid, L. (2021). Anonymisation of Medical Records Using Hybrid NER and Rule-Based Systems. Journal of Biomedical Semantics, 12(1), 1–19.

[20] Liu, Y., Chen, D., & Ma, L. (2023). Font Attribute Recognition from Natural Scene Images Using Convolutional Networks. Pattern Recognition Letters, 168, 55–62.

[21] Matas, J., Chum, O., Urban, M., & Pajdla, T. (2021). Robust Wide-Baseline Stereo and Scene Text Detection: A Unified View. International Journal of Computer Vision, 129(4), 1062–1085.

[22] Minaee, S., Boykov, Y., Porikli, F., & Terzopoulos, D. (2022). Image Segmentation Using Deep Learning: A Survey. IEEE Transactions on Pattern Analysis and Machine Intelligence, 44(7), 3523–3542.

[23] Nguyen, T., Nguyen, B., & Tran, D. (2022). A Comprehensive Study of Text Detection Algorithms for Screen Content Images. Image and Vision Computing, 121, 104418.

[24] Otsu, N. (1979). A Threshold Selection Method from Gray-Level Histograms. IEEE Transactions on Systems, Man, and Cybernetics, 9(1), 62–66.

[25] Pizer, S. M., Amburn, E. P., Austin, J. D., Cromartie, R., Geselowitz, A., Greer, T., & Zimmerman, J. B. (1987). Adaptive Histogram Equalization and Its Variations. Computer Vision, Graphics, and Image Processing, 39(3), 355–368.

[26] Rao, K., Prasad, A., & Nair, V. (2023). Automated Aadhaar Number Masking in Scanned Documents Using Deep Learning. International Journal of Intelligent Systems and Applications, 15(2), 33–44.

[27] Redmon, J., & Farhadi, A. (2020). YOLOv4 and Its Applications to Scene Text Detection in Real-Time. Neural Networks, 127, 142–153.

[28] Shi, B., Bai, X., & Yao, C. (2021). An End-to-End Trainable Neural Network for Image-Based Sequence Recognition. IEEE Transactions on Pattern Analysis and Machine Intelligence, 39(11), 2298–2304.

[29] Smith, R. (2007). An Overview of the Tesseract OCR Engine. Proceedings of the 9th International Conference on Document Analysis and Recognition (ICDAR), 629–633.

[30] Stuchi, J. A., Brun, A. L., & Bertolini, D. (2017). Screen Text Detection for Real-Time Caption Extraction in Live Video Broadcasting. IEEE MultiMedia, 24(3), 56–67.

[31] Tomasi, C., & Manduchi, R. (1998). Bilateral Filtering for Gray and Color Images. Proceedings of the IEEE International Conference on Computer Vision (ICCV), 839–846.

[32] Wang, K., Babenko, B., & Belongie, S. (2022). End-to-End Scene Text Recognition in the Wild. International Journal of Computer Vision, 130(9), 2341–2359.

[33] Yang, H., Kim, J., Lee, S., & Park, J. (2022). Privacy-Preserving Document Image Anonymisation via GAN-Based Inpainting. IEEE Access, 10, 15824–15836.

[34] Ye, Q., & Doermann, D. (2022). Text Detection and Recognition in Imagery: A Survey. IEEE Transactions on Pattern Analysis and Machine Intelligence, 37(7), 1480–1500.

[35] Zhang, Y., Gu, L., & Liu, C. (2023). Real-Time OCR for Privacy-Sensitive Documents on Mobile Platforms. Multimedia Tools and Applications, 82(6), 8741–8762.

[36] Zhao, X., Lin, K., & Fu, H. (2022). Towards Zero-Latency PII Redaction in Streaming Video: A Pipeline Architecture. Journal of Information Security and Applications, 67, 103198.

[37] Zhou, X., Yao, C., Wen, H., Wang, Y., Zhou, S., He, W., & Liang, J. (2017). EAST: An Efficient and Accurate Scene Text Detector. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 5551–5560.

[38] Zuiderveld, K. (1994). Contrast Limited Adaptive Histogram Equalization. In P. Heckbert (Ed.), Graphics Gems IV (pp. 474–485). Academic Press.